DOM (Document Object Model)

The DOM represents the HTML page as a tree of objects. Mastering how to select, traverse, and manipulate DOM nodes is essential for exams, interviews, and real-world JavaScript work.

Core concepts:

- DOM = tree representation of the document
- Every part of the page is a Node
- `document` is the entry point

```
document                    // root object
document.body               // <body>
document.head               // <head>
document.documentElement    // <html>
```

Core concepts:

- Element → HTML tag (`<div>` , `<p>`)→ `nodeType === 1`
- Text → text inside elements → `nodeType === 3`
- Comment → `<!-- -->` → `nodeType === 8`
- Document → `document` → `nodeType === 9`

```
node.nodeType
node.nodeName          // "DIV", "#text"
node.tagName           // "DIV" (Element only)
```

Core actions - Selecting elements:

- querySelector(selector) → First matching element or `null`

```
document.querySelector('.card')
```

- querySelectorAll(selector) → All matches

```
document.querySelectorAll('.card')
```

- getElementById(id) → Element or `null`

```
document.getElementById('main')
```

- getElementsByClassName(class) → Live HTMLCollection

```
document.getElementsByClassName('item')
```

- getElementsByTagName(tag) → Live HTMLCollection

```
document.getElementsByTagName('div')
```

Traversing - Node vs Element:

💡 Whitespace creates Text nodes. Prefer Element-based APIs.

Node traversal (includes text & comments)

- parentNode
- firstChild / lastChild
- childNodes
- nextSibling / previousSibling

```
node.childNodes
node.firstChild
```

Element traversal (recommended)

- parentElement
- firstElementChild / lastElementChild
- children
- nextElementSibling / previousElementSibling

```
el.children
el.nextElementSibling
```

Traversing - Practical patterns

- Parent → children

```
el.children
```

- Child → parent

```
el.parentElement
```

- Siblings

```
el.nextElementSibling
```

- Closest ancestor (including self)

```
el.closest('.container')
```

Search & Test

- matches(selector) → Does this element match?

```
el.matches('.active') // true | false
```

- closest(selector) → Nearest ancestor or `null`

```
el.closest('form')
```

Text & HTML content

- textContent → Raw text (safe, fast)

```
el.textContent = 'Hello'
```

- innerHTML → Parsed HTML (⚠ injection risk)

```
el.innerHTML = '<strong>Hello</strong>'
```

- innerText → Rendered text (CSS/layout dependent)

Create, Insert & Remove

- createElement(tag) → Create element

```
const div = document.createElement('div')
```

- append / prepend → Insert inside

```
parent.append(child)
```

- before / after → Insert relative

```
ref.before(el)
```

- replaceWith → Replace node

```
ref.replaceWith(el)
```

- emove → Remove node

```
el.remove()
```

Clone & Move

- loneNode(deep) → Clone element

```
el.cloneNode(true)
```

- append existing node → Moves it (not copied)

```
otherParent.append(el)
```

Loops & Iteration

- for...of → Elements / NodeList

```
for (const el of nodeList) {}
```

- forEach → NodeList only

```
nodeList.forEach(el => {})
```

- classic for → Full control

```
for (let i = 0; i < el.children.length; i++) {}
```